# Week 14: Redis caching and choosing a database

**Learning outcomes:** CLO4, CLO5

## Lesson

- Redis keeps data in memory and looks it up by key: very fast. Types: string, hash, list, set, sorted set.
- **TTL:** key expires after n seconds.
- **Cache-aside:** check cache → **miss**: read MySQL, store with TTL, return → **hit**: return cached value. MySQL stays the main copy.
- Plan for data changes (delete or refresh the key) and for Redis being down (read MySQL).
- Use codes as keys, names as values, `area_km2` as sorted-set score. No users or sessions in this data.
- Pick per job: MySQL (related data, rules), MongoDB (flexible documents), Redis (fast, temporary), graph DB (links are the question).

## Worked example

Run the cache app in apps/ (`apps/README.md`) twice with the same code, then answer in `work.md`.

## The cache for this week

**Cache key:** `fese304:districts:<province_pcode>` (e.g. `fese304:districts:KH01`). Value: JSON array of `district_pcode`, `district_name`. TTL: 60 s.

**Your keys (tasks 3–4):** create them yourself and start their names with `fese304:lab14:`.

Tables: data guide (`data/README.md`).

## Lab tasks

1. Run `cache.py` or `cache.mjs` twice with one code; save the miss and the hit.
2. In redis-cli, view the JSON and TTL. Wait for expiry; run again for a miss.
3. Build a hash, a list and a set from province or district fields.
4. Build a sorted set of provinces by area; compare with MySQL ORDER BY. What does Redis do with equal scores?
5. Delete the app's key from task 1 (`DEL fese304:districts:<code>`), run the app again and save the miss. Write a policy for data changes and Redis downtime.

## Homework

1. Submit Redis commands, output, expiry notes and cache code.
2. For each capstone access pattern, choose MySQL, MongoDB or Redis and say why.
3. Why is Redis fast? Give one case where a graph database fits better.

## Capstone sprint

**C14 / Checkpoint 4:** Redis feature, NoSQL model and a demo rehearsal.

## Submit

Answers in `work.md`, reflection in `reflection.md`, output in `results/`. Commit and push. See SUBMISSIONS.md.